GEM: An implementation of the ghost-Gutzwiller approximation for simulating interacting quantum systems

Samuele Giuli [1⋆], Tsung-Han Lee [2,3], Yong-Xin Yao [4,5], Ina Park [1], Harrison LaBollita [1], Ivan Pasqua [1], Nils Wentzell [1], Nicola Lanatà [1,6] and Olivier Gingras [1,7†]

1 Center for Computational Quantum Physics, Flatiron Institute, 162 Fifth Avenue, New York, New York 10010, USA
2 Department of Physics, National Chung Cheng University, Chiayi 62102, Taiwan
3 Physics Division, National Center for Theoretical Sciences, Taipei 10617, Taiwan
4 Ames National Laboratory, Ames, Iowa 50011, USA
5 Department of Physics and Astronomy, Iowa State University, Ames, Iowa 50011, USA
6 School of Physics and Astronomy, Rochester Institute of Technology, 84 Lomb Memorial Drive, Rochester, New York 14623, USA
7 Université Paris-Saclay, CNRS, CEA, Institut de physique théorique, 91191, Gif-sur-Yvette, France

⋆ sgiuli@flatironinstitute.org , † ogingras@flatironinstitute.org

arXiv:2609.32633v2 [cond-mat.str-el] 13 Oct 2026

Abstract

We present GEM (Ghost Embedding Method), an open-source software package written in Python for computing equilibrium properties of strongly correlated electronic systems within the ghost-Gutzwiller approximation method. GEM provides a computationally efficient framework for studying multi-orbital lattice models. It supports zero- and finite-temperature calculations and symmetry broken phases. It is integrated with the TRIQS ecosystem, providing tools for model construction, self-consistent solution, and evaluation of physical observables. We first detail the method's theoretical formulation, then we present the software architecture, and finally we introduce some practical workflow, which also validates the implementation against established results. In particular, we illustrate the capabilities of GEM through multiorbital and finite-temperature applications and discuss its computational cost relative to more demanding quantum embedding approaches.

Copyright attribution to authors.
This work is a submission to SciPost Physics Codebases.
License information to appear upon publication.
Publication information to appear upon publication.

Received Date
Accepted Date
Published Date

Contents

1 Introduction 2

2 Installation 4

3 Method 5
3.1 Interacting lattice problem 5
3.2 Dynamical functional and ghost embedding 6

3.3 Saddle point of the ghost embedding functional 7
3.4 Self-consistency cycle 8

4 Implementation 8
4.1 `Lattice` and `Fragment` objects 9
4.2 Solvers: the `gemSolver` class, `SimpleED` and generic template 10
4.3 Testing suit 11

5 Examples 12
5.1 Bethe lattice: thermodynamics 12
5.2 Square lattice: magnetic phase diagram 14
5.3 Multi-orbital Bethe lattice: Hund's physics 17

6 Outlook 20

7 Conclusions 21

A Density-matrix fitting 22

References 24

1 Introduction

Nonperturbative calculations for correlated-electron models require a compromise between physical fidelity, numerical cost, and the range of accessible observables. Quantum embedding methods provide one of the main routes for treating strong local correlations while retaining a tractable computational problem. Among those, dynamical mean-field theory (DMFT) [1] played a central role in the understanding of the Mott transition and the effects of strong correlations. Its self-consistency loop requires repeated solutions of an interacting quantum impurity problem and the evaluation of frequency-dependent quantities, namely the impurity Green's function. In most cases of interest, obtaining these solutions quickly becomes numerically prohibitive as one increases the amount of orbitals, lower the temperature, wants to perform broad parameter scans or is interested in symmetry-broken phases.

The conventional Gutzwiller approximation (GA) is a substantially lighter approach which provides a good description of the renormalized low-energy quasiparticles but does not reproduce the incoherent high-energy Hubbard bands [2]. One of its extension, the ghost-Gutzwiller approximation (ghost-GA) [3], consists of augmenting the variational auxiliary space in order to recover high-energy features in the spectral function and to reach DMFT-like total energies and static expectation values already with a moderate enlargement of the auxiliary space [3,4]. Recently, an approach based on a free-energy functional has proven the equivalence between DMFT and ghost-GA in the limit of infinite auxiliary states, providing a route to extend ghost-GA to perform finite temperature calculations [5], yet at the moment, no open-source software provides an implementation of this formulation.

GEM addresses this gap by making the systematically improvable ghost-GA formulation, both at zero- and finite-temperature, available within an open-source research software. The implementation follows the free-energy functional that unifies zero-temperature ghost-GA, its finite-temperature extension, and DMFT: stationary conditions are expressed using static or thermal expectation values of effective Hamiltonians. DMFT is obtained as the number of

auxiliary bath modes per physical orbital, B, tends to infinity: $B \to \infty$ [5]. Although the exact correspondence holds in the $B \to \infty$ limit, convergence can be rapid in practice [4]. For the finite-temperature Hubbard-model benchmarks reported in Ref. [5], calculations with only $B = 3$ already reproduce DMFT thermodynamic results with high accuracy across the investigated interaction and temperature regimes. Moreover, already the much simpler GA (ghost-GA with $B = 1$) gives a good description of the thermodynamic stability of symmetry broken phases, and provides a good estimate for total energies and other static observables like order parameters [6–8].

The software ecosystem for quantum embedding is by now well developed, particularly for DMFT, where mature implementations exist for a broad range of impurity solvers and many-body workflows. These include exact-diagonalization [9–12], continuous-time quantum Monte Carlo [13–20], tensor-network [21, 22], and numerical renormalization group approaches [23], together with broader frameworks such as TRIQS [24] and ALPS [25] that provide reusable infrastructure for Green's functions, operators, data handling, and solver interoperability. This maturity makes DMFT a flexible and powerful tool, but many applications do not require the full cost and complexity of a fully dynamical impurity calculation.

Gutzwiller-based approaches provide a complementary route in this regime. Publicly available implementations of the Gutzwiller approximation and ghost-GA exist, such as CyGutz [26] and Portobello [27], but no maintained open-source package implements the finite-temperature ghost-GA formulation recently described in Ref. [5]. GEM addresses this gap by providing a variational embedding hierarchy controlled by the number B of auxiliary bath modes (ghost orbitals). Increasing B systematically enlarges the auxiliary space and approaches the DMFT limit, while small values of B can already provide an advantageous balance between computational cost and accuracy [5]. GEM therefore occupies an intermediate regime between inexpensive static Gutzwiller approximations and fully dynamical DMFT workflows, enabling ground-state and thermodynamic calculations without requiring a full frequency-dependent impurity solution at every stage.

As a result, GEM adopts a package design that implements the ghost-embedding algorithm and its core data structures, while remaining interoperable with external numerical and impurity-solver components. At the moment, it comes with an internal ED impurity solver, but it also offers a template to generate external impurity solvers. In the future, impurity solvers based on various numerical methods will be interfaced, providing seamless access to a variety of options depending on the physical systems of interest. This positioning of GEM enables two complementary workflows: 1) It can be used as a lower-cost embedding method for static and thermal observables at moderate auxiliary-space size, for which it has been shown that the agreement with DMFT is already excellent at small bath sizes [4], and 2) converged ghost-GA solutions can be used as a starting point for DMFT as shown in Ref. [28].

The target users are researchers developing or applying quantum embedding methods in condensed matter physics and materials theory. Representative uses include the study of Hubbard models and interacting multiorbital systems, phase-diagram calculations, tests of convergence with auxiliary-space size, investigations of Mott and orbital-selective transitions, and benchmarking or developing impurity solvers. By providing the method as a reusable package rather than a collection of project-specific scripts, GEM supports reproducible comparisons across models, solvers, temperatures, and approximation levels.

This manuscript is structured as follows. In Sec. 2, we provide information on how to install the software. In Sec. 3, we give a brief description of the method, followed in Sec. 4 by a description of its implementation in GEM. In Sec. 5, we present three examples of prototypical calculations illustrating the different capabilities of the software: 1) finite temperature calculations of a single-orbital model for total energy and free energy, 2) finite temperature calculations of a single-orbital model for the phase diagrams of symmetry broken phases, and 3) zero

temperature calculations of a multi-orbital Hund system for various interaction strengths. In Sec. 6, we provide an outlook on future directions for the development of the software and in Sec. 7, we summarize the content of the manuscript.

2 Installation

Since GEM is pure Python, building and installing it requires no additional compilation. Its only dependencies are shown in Table 1. These dependencies are a Python 3 interpreter, CMake, and four scientific Python packages: NumPy, SciPy, Numba and h5py. The versions used to test the current release are listed in Table 1. These versions are known to work with GEM but should not be interpreted as minimum version requirements. Since each Numba release supports a bounded range of NumPy versions, the two should be upgraded together.

Software	Ref.	Tested
Python	[29]	3.9.6
CMake	[30]	4.3.2
NumPy	[31]	2.0.2
SciPy	[32]	1.13.1
Numba	[33]	0.60.0
h5py	[34]	3.14.0

Table 1: External software dependencies of GEM along with versions known to work.

Optionally, GEM offers an MPI-based parallelization scheme using mpi4py [35]. Details of the parallelization are provided in Section 4. If mpi4py is not installed, GEM falls back to a serial calculation, with parallelism limited to any multithreading provided by the LAPACK and BLAS libraries linked to the installed Python packages.

The internal `SimpleED` solver requires nothing beyond these packages; interfaces to external impurity solvers require their corresponding libraries, which should be cited alongside GEM when used. Some examples additionally use Matplotlib [36] for plotting and TRIQS [24] for lattice construction. The package is built and installed out of source with CMake:

```
git clone https://github.com/TRIQS/gem.git
mkdir gem.build && cd gem.build
cmake ../gem
make
make test
make install
```

By default the installation prefix is the one associated to the Python interpreter found by CMake—typically the interpreter of an active virtual environment—so that `import gem` works directly after installation, and the test suite is built and run by `make test`. The configuration step accepts optional flags, passed as `-D<Option>=<value>` on the `cmake` line, in any number and order. The available flags as listed in Table 2.

Option	Effect
`-DCMAKE_INSTALL_PREFIX=<path>`	install into `<path>` instead of the Python prefix
`-DBuild_Tests=OFF`	skip the test suite (`ON` by default)
`-DBuild_Documentation=ON`	build documentation (`OFF` by default; requires Sphinx [37])

Table 2: Options that can be passed when calling `cmake`.

3 Method

GEM implements ghost-GA in the quantum-embedding formulation introduced in Refs. [3,5]. In this section, we briefly summarize the formulation used in the code, with particular emphasis on the quantities entering the numerical self-consistency. For a more complete derivation and a discussion of the connection between ghost-GA and DMFT, we refer the reader to Ref. [5].

3.1 Interacting lattice problem

We consider a general multi-orbital lattice Hamiltonian

$$\hat{H} = \hat{H}_0 + \hat{H}_{\text{int}}, \tag{1}$$

with a local interaction

$$\hat{H}_{\text{int}} = \sum_{i=1}^{\mathcal{N}} \hat{H}^i_{\text{int}}, \tag{2}$$

and a one-body contribution

$$\hat{H}_0 = \sum_{i,j=1}^{\mathcal{N}} \sum_{\alpha=1}^{\nu_i} \sum_{\beta=1}^{\nu_j} [h_0]_{i\alpha,j\beta} \hat{c}^\dagger_{i\alpha} \hat{c}_{j\beta}. \tag{3}$$

Here, $\hat{c}^\dagger_{i\alpha}$ ($\hat{c}_{i\alpha}$) is an operator that creates (annihilates) an electron on the correlated fragment labeled by i (often site) with physical fermionic degrees of freedom labeled by α. We have $i = 1,\ldots,\mathcal{N}$ where $\mathcal{N}$ is the total number of correlated fragments, and $\alpha = 1,\ldots,\nu_i$ where ν_i is the number of spin-orbital electronic levels in the fragment i. h_0 is a matrix of scattering amplitudes.

In the present form of the code, we only consider cases for which the intra-fragment interaction is two-body and can be expressed as:

$$\hat{H}^i_{\text{int}} = \sum_{i=1}^{\mathcal{N}} \sum_{\alpha,\beta,\gamma,\delta=1}^{\nu_i} \frac{U^i_{\alpha\beta\gamma\delta}}{2} \hat{c}^\dagger_{i\alpha} \hat{c}_{i\beta} \hat{c}^\dagger_{i\gamma} \hat{c}_{i\delta}, \tag{4}$$

but the derivation is completely general. It is convenient to separate the local intra-fragment one-body contribution from the inter-fragment hopping,

$$h_0 = \epsilon + t, \tag{5}$$

where $\epsilon = \text{diag}(\epsilon_1,\ldots,\epsilon_{\mathcal{N}})$, or $[\epsilon]_{i\alpha,j\beta} = [h_0]_{i\alpha,i\beta}\delta_{ij} \equiv [h^i_0]_{\alpha\beta}\delta_{ij}$, contains the local one-body blocks and t contains the off-diagonal hopping terms. The local one-body contribution can then be combined with the interaction into the local Hamiltonian

$$\hat{H}^i_{\text{loc}} = \sum_{\alpha,\beta=1}^{\nu_i} [\epsilon_i]_{\alpha\beta} \hat{c}^\dagger_{i\alpha} \hat{c}_{i\beta} + \hat{H}^i_{\text{int}}. \tag{6}$$

The full Hamiltonian can therefore be written as

$$\hat{H} = \sum_{i\neq j} \sum_{\alpha=1}^{\nu_i} \sum_{\beta=1}^{\nu_j} [t_{ij}]_{\alpha\beta} \hat{c}^\dagger_{i\alpha} \hat{c}_{j\beta} + \sum_i \hat{H}^i_{\text{loc}}. \tag{7}$$

When the system is translationally invariant, with a primitive unit cell that contains one or more fragments, one can rewrite the Hamiltonian as follows:

$$\hat{H} = \sum_{I,J} \sum_{i\in I, j\in J} \sum_{\alpha=1}^{\nu_i} \sum_{\beta=1}^{\nu_j} [t^{IJ}_{ij}]_{\alpha\beta} \hat{c}^\dagger_{Ii\alpha} \hat{c}_{Jj\beta} + \sum_I \sum_{i\in I} \hat{H}^{Ii}_{\text{loc}}. \tag{8}$$

Here, I (J) is the index of a superlattice cell and i (j) is index running over the fragments belonging to a superlattice cell. Transforming the kinetic part of Eq. (8) to reciprocal space, we get the following Hamiltonian:

$$\hat{H} = \sum_{K} \sum_{i,j} \sum_{\alpha=1}^{\nu_i} \sum_{\beta=1}^{\nu_j} [t_{ij}^{K}]_{\alpha\beta} \hat{c}_{Ki\alpha}^{\dagger} \hat{c}_{Kj\beta} + \sum_{I} \sum_{i \in I} \hat{H}_{\mathrm{loc}}^{Ii}. \tag{9}$$

where K is a reciprocal lattice vector belonging to the Brillouin zone of the superlattice, t^K is the Fourier transformed hopping matrix and $\hat{c}_K^{\dagger}$ ($\hat{c}_K$) are the Fourier transformed creation (annihilation) operators.

3.2 Dynamical functional and ghost embedding

The starting point of the formulation is the finite-temperature dynamical functional

$$\begin{aligned} \mathcal{L}_{\beta}[\Sigma_i, \Delta_i] = &-\beta^{-1} \sum_{n} e^{i\omega_n 0^+} \mathrm{Tr} \ln \left[i\omega_n \mathbf{1} - h_0 - \Sigma(i\omega_n) \right] \\ &+ \beta^{-1} \sum_{i} \sum_{n} e^{i\omega_n 0^+} \mathrm{Tr} \ln \left[i\omega_n \mathbf{1} - \epsilon_i - \Delta_i(i\omega_n) - \Sigma_i(i\omega_n) \right] \\ &+ \sum_{i} \Omega_{\mathrm{imp}}^{i}[\Delta_i], \end{aligned} \tag{10}$$

where $\Sigma = \mathrm{diag}(\Sigma_1, \ldots, \Sigma_{\mathcal{N}})$ is a block matrix, Σ_i is the self-energy of fragment i which contains all the intra-fragment electronic correlations, Δ_i is the hybridization function associated with fragment i, and $\Omega_{\mathrm{imp}}^{i}$ is the free energy of the corresponding interacting impurity problem. We work in imaginary frequencies, with ω_n the n$^{\mathrm{th}}$ Matsubara frequency, and β is the inverse temperature considered. As demonstrated in Ref. [5], the stationary conditions of the functional in Eq. (10) are the DMFT self-consistency conditions [1].

Given a positive integer B, which quantifies the number of ghost degrees of freedom and therefore the size of the variational space, one can introduce [5] the following parametrization for the self-energies and hybridization functions of fragment i:

$$\Sigma_i(z) = z\mathbf{1}_{\nu_i} - \left[\mathcal{R}_i^{\dagger} \left(z\mathbf{1}_{B\nu_i} - \Lambda_i \right)^{-1} \mathcal{R}_i \right]^{-1} - \epsilon_i, \tag{11}$$

$$\Delta_i(z) = \mathcal{D}_i^{T} \left(z\mathbf{1}_{B\nu_i} + \Lambda_i^{c} \right)^{-1} \mathcal{D}_i^{*}. \tag{12}$$

where $\Lambda_i, \Lambda_i^c \in \mathbb{C}_{B\nu_i \times B\nu_i}$ are Hermitian matrices, $R_i^{\dagger}, \mathcal{D}_i \in \mathbb{C}_{B\nu_i \times \nu_i}$, and $z \in \mathbb{C}$. Within this parametrization, the functional recover the ghost-GA functional at zero temperature and provide a natural finite temperature extension, as shown in Ref. [5]. Here, B is a parameter that determines the enlargement of the auxiliary space with respect to the physical one. With $B = 1$, the auxiliary space and the physical space have the same dimensionality and at zero temperature, one recovers the standard GA. For $B > 1$ the auxiliary space is larger than the physical one and these additional degrees of freedom, known as *ghost* orbitals, increase the expressivity of the parametrization of self-energies and hybridization functions, and one recovers DMFT in the $B \to \infty$ limit. This pole expansion representation of the self-energies and hybridization functions can be interpreted as an hybridization with fictitious auxiliary fermionic states. This way the new ghost embedding functional is given by:

$$\mathcal{L}_{\beta}[\{\mathcal{D}_i\}, \{\Lambda_i^c\}, \{\mathcal{R}_i\}, \{\Lambda_i\}] = \Omega_{\mathrm{qp}}[\mathcal{R}, \Lambda] + \sum_{i=1}^{\mathcal{N}} \Omega_{\mathrm{emb}}^{i}[\mathcal{D}_i, \Lambda_i^c] - \sum_{i=1}^{\mathcal{N}} \Omega_{0,\mathrm{emb}}^{i}[\mathcal{D}_i, \Lambda_i^c; \mathcal{R}_i, \Lambda_i] \tag{13}$$

where $\Omega_{\rm qp}$, $\Omega^i_{\rm emb}$ and $\Omega^i_{0,\rm emb}$ are grand canonical potentials of systems identified by the following Hamiltonians:

$$\hat{H}_{\rm qp} = \sum_{ij}\sum_{a=1}^{B\nu_i}\sum_{b=1}^{B\nu_j} \hat{f}^\dagger_{ia}\left(\delta_{ij}[\Lambda^i]_{ab} + \sum_{\alpha\beta} R_{i,a\alpha}\, t^{ij}_{\alpha\beta}\, R^\dagger_{j,\beta b}\right)\hat{f}_{jb}, \tag{14}$$

$$\begin{aligned}\hat{H}^i_{\rm emb} = &\sum_{\alpha,\beta=1}^{\nu_i}[\epsilon_i]_{\alpha\beta}\hat{c}^\dagger_{i\alpha}\hat{c}_{i\beta} + \hat{H}^i_{\rm int}[\hat{c}^\dagger_i,\hat{c}_i] \\ &+\sum_{a=1}^{B\nu_i}\sum_{\alpha=1}^{\nu_i}\left([\mathcal{D}^i]_{a\alpha}\,\hat{b}^\dagger_{i,a}\hat{c}_{i,\alpha} + \text{h.c.}\right) + \sum_{\alpha\beta=1}^{B\nu_i}[\Lambda^i_c]_{\alpha\beta}\,\hat{f}^\dagger_\alpha\hat{b}_\beta,\end{aligned} \tag{15}$$

$$\hat{H}^i_{0,\rm emb} = \sum_{a,b=1}^{B\nu_i}\left[\Lambda^i\right]_{ab}\hat{\tilde{f}}^\dagger_{ia}\hat{\tilde{f}}_{ib} + \sum_{a,b=1}^{B\nu_i}\left(\left[\mathcal{D}^i(R^i)^T\right]_{ab}\hat{\tilde{f}}^\dagger_{ib}\hat{\tilde{b}}_{ia} + \text{H.c.}\right) + \sum_{a,b=1}^{B\nu_i}\left[\Lambda^i_c\right]_{ab}\hat{\tilde{b}}_{ib}\hat{\tilde{b}}^\dagger_{ia}. \tag{16}$$

where δ_{ij} is Dirac's delta. Here, the different Hamiltonians encode the original physical fermions in different ways. In Eq. (15), the original physical fermionic fields are represented by the $\hat{c}_\alpha$ fermions. In Eqs. (14, 16) instead, the original physical fermionic fields are represented by linear combinations of the other fermionic fields, respectively by $\sum_a R^\dagger_{\alpha a}\hat{f}_a$ and $\sum_a R^\dagger_{\alpha a}\hat{\tilde{f}}_a$, more details on the role of these Hamiltonians can be found in Ref. [5].

3.3 Saddle point of the ghost embedding functional

Recognizing that the ghost embedding functional can be interpreted as a sum of the grand canonical potentials drastically simplifies the saddle point conditions with respect to the four set of matrices $\{\mathcal{R}^i,\Lambda^i,\mathcal{D}^i,\Lambda^i_c\}_{\forall i}$. These conditions are:

$$\frac{\partial\mathcal{L}_\beta}{\partial[\mathcal{R}_i]_{a\alpha}} = 0 \Rightarrow \sum_{j=1}^{\mathcal{N}}\sum_{\beta=1}^{\nu_j}\sum_{b=1}^{B\nu_j}[t^{ij}]_{\alpha\beta}[\mathcal{R}^\dagger_j]_{\beta b}\langle\hat{f}^\dagger_{ia}\hat{f}_{jb}\rangle_{\rm qp} = \sum_{b=1}^{B\nu_i}[\mathcal{D}_i]_{b\alpha}\langle\hat{\tilde{f}}^\dagger_{ia}\hat{\tilde{b}}_{ib}\rangle_{0,\rm emb,i}\,, \tag{17}$$

$$\frac{\partial\mathcal{L}_\beta}{\partial[\Lambda_i]_{ab}} = 0 \Rightarrow \langle\hat{f}^\dagger_{ia}\hat{f}_{ib}\rangle_{\rm qp} = \langle\hat{\tilde{f}}^\dagger_{ia}\hat{\tilde{f}}_{ib}\rangle_{0,\rm emb,i}\,, \tag{18}$$

$$\frac{\partial\mathcal{L}_\beta}{\partial[\mathcal{D}_i]_{a\alpha}} = 0 \Rightarrow \langle\hat{c}^\dagger_{i\alpha}\hat{b}_{ib}\rangle_{\rm emb,i} = \langle\Big(\sum_{a=1}^{B\nu_i}[\mathcal{R}_i]_{a\alpha}\hat{\tilde{f}}^\dagger_{ia}\Big)\hat{\tilde{b}}_{ib}\rangle_{0,\rm emb,i}\,, \tag{19}$$

$$\frac{\partial\mathcal{L}_\beta}{\partial[\Lambda^c_i]_{ab}} = 0 \Rightarrow \langle\hat{b}_{ia}\hat{b}^\dagger_{ib}\rangle_{\rm emb,i} = \langle\hat{\tilde{b}}_{ia}\hat{\tilde{b}}^\dagger_{ib}\rangle_{0,\rm emb,i}\,. \tag{20}$$

The saddle point equations are now expressed as matching conditions for values of thermal one-body density matrices belonging to the different problems. Equations (17-20) are the self-consistency equations of the ghost embedding functional and $\langle\hat{A}\rangle_X$ is the thermal expectation value of operator $\hat{A}$ given the Hamiltonian $\hat{H}_X$ with X specifying either the quasiparticle Hamiltonian or one of the embedding Hamiltonians, more specifically $\langle\hat{A}\rangle_X \equiv \text{Tr}\left[\hat{\rho}_X\hat{A}\right]$ with $\hat{\rho}_X \equiv \frac{e^{-\beta\hat{H}_X}}{\text{Tr}\left[e^{-\beta\hat{H}_X}\right]}$. When a bath size is finite, the advantage with respect to DMFT is evident: since the self-consistency is based on thermal one-body density matrices, there is no need to compute any Green's function. This not only simplifies the self-consistency itself, since it is rooted in static expectation values instead of dynamical objects like Green's functions, but also allows the use of variational impurity solvers that have access to those thermal expectation values, such as DMRG [38], tensor networks solvers, or approaches that can reduce the Hilbert space to the target manyfold of interest [39].

3.4 Self-consistency cycle

The self-consistency cycle of the ghost embedding functional is defined by the simultaneous fulfillment of the saddle-point Eqs. (17-20). We implement the self-consistency by imposing some of them at each step of the self-consistency until convergence is realized. The cycle is schematically depicted in Figure 1 and can be decomposed in the following steps:

1. Solve $\hat{H}_{\text{qp}}$ defined in Eq. (14) to compute the left-hand side of Eqs. (17, 18).
2. Update $\Delta^i[\mathcal{D}^i, \Lambda_c^i]$ to fit the right-hand side of Eqs. (17, 18) to the updated left side.
3. Solve $\hat{H}^i_{\text{emb}}$ in Eq. (15) to compute the left-hand side of Eqs. (19, 20).
4. Update $\Sigma^i[\mathcal{R}^i, \Lambda^i]$ to fit the right-hand side of Eqs. (19, 20) to the updated left side.

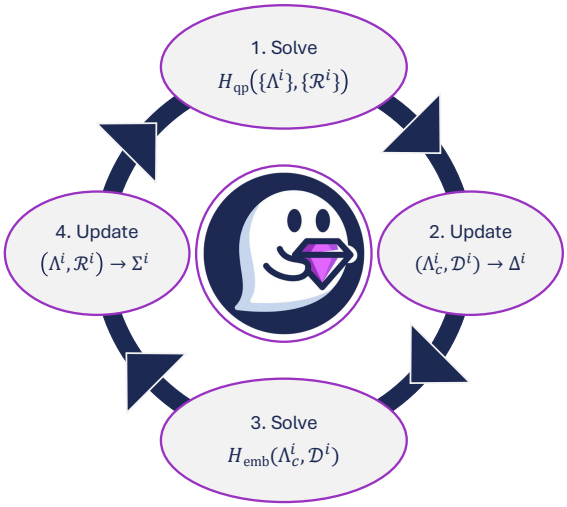

Figure 1: Schematic representation of the self-consistency cycle of the finite temperature ghost-GA implementation present in GEM with the logo of the software at the center.

The self-consistency is reached when the matrices are converged within the tolerance set by the user and all the saddle point equations are satisfied. At finite temperature, steps 4 and 2 are performed by a fitting routine for the thermal density matrices. Details on how the thermal density-matrix fitting is performed are given in Appendix A. At zero temperature, as shown in Ref. [5], one can fix the relative gauge between the $\hat{f}$ and $\hat{b}$ fermions, and the corresponding matching conditions are realized by the ghost-GA saddle point equations. In that case, no thermal density-matrix fitting is needed.

4 Implementation

GEM is implemented as a Python package. Distributed-memory parallelism is provided through a message passing interface (MPI) layer, gathered in a single `mpi.py` module that both parallel layers of the code share: the distribution of the k-point sums in the quasiparticle problem, provided by the `Lattice` object (See Sec. 4.1), and the distribution of the symmetry sectors in the exact diagonalization of the embedding Hamiltonian, provided by the internal `SimpleED` solver (See Sec. 4.2). The module resolves the communicator, splits the work among the ranks and reduces the partial results, so that the physics routines are written once and are agnostic

of the number of processes; the same code path is taken by a serial run, which falls back on a single-rank calculation when mpi4py [35] is not available or if MPI is deactivated. On top of this, parallelism is also inherited from the threaded `BLAS` [40] and `LAPACK` [41] implementations that NumPy and SciPy link against. The number of threads is controlled through the usual environment variables, e.g. `OMP_NUM_THREADS`.

The main software components closely follow the decomposition of the ghost-embedding functional introduced in Sec. 3.2. In particular, the lattice quasiparticle problem and the local embedding problems are represented by separate objects, called `Lattice` and `Fragment`, presented in Sec. 4.1, while the impurity solver is exposed through an independent interface, presented in Sec. 4.2. This separation allows the equations defining the ghost-GA saddle point to remain independent of the particular method used to solve the interacting embedding Hamiltonian.

4.1 `Lattice` and `Fragment` objects

The two core classes around which the software is structured are the `Lattice` and the `Fragment` classes. The `Fragment` class contains all the information that is specific to that fragment, for example the fragment i will store the matrices Λ_i^c and $\mathcal{D}_c$ that parametrize the hybridization function Δ_i, the matrices Λ_i and $\mathcal{R}_i$ that parametrize the self-energy Σ_i, and the intra-fragment local Hamiltonian Eq. (6). It solves the embedding problems Eqs. (15, 16) and it computes local expectation values of observables. The `Lattice` object instead contains the inter-fragment non-local kinetic terms of the Hamiltonian in Eq. (7) and solves the quasiparticle problem Eq. (14).

The `Fragment` class. At initialization time, an object belonging to this class, for a fragment labeled by i, will require the inputs listed in Table 3. We also list the corresponding physical quantity introduced in prior sections, when applicable. The only two arguments in the Table that were not described before are: 1) `solver`, that is the impurity solver used to solve the embedding Hamiltonian (15) and is explained in more details in Sec. 4.2, and 2) `verbose`, a variable used to tune the verbosity of the printing from little (`verbose=0`) to very verbose (`verbose=3`).

Argument	type	Corresponding quantity
`nimp`	`int`	ν_i
`nbath`	`int`	$B\,\nu_i$
`eloc`	`np.ndarray`	ϵ_i
`Utensor`	`np.ndarray`	$U^i_{\alpha\beta\gamma\delta}$
`solver`	See Sec. 4.2	
Optional argument	**type**	**Corresponding quantity**
`Lambda`	`np.ndarray`	Λ_i
`R`	`np.ndarray`	$\mathcal{R}_i$
`Lambda_c`	`np.ndarray`	Λ_i^c
`D`	`np.ndarray`	$\mathcal{D}_i$
`verbose`	`int`	

Table 3: Inputs used to initialize a `Fragment` object.

The `Fragment` objects are then also responsible for the update of the hybridization function and self-energy parameters, e.g. steps 2 and 4 in Sec. 3.4 and Fig. 1. This is done via two methods of this object, namely `update_self_energy` and `update_hybridization` that automatically update the internal variables of the `Fragment`. At finite temperature, these

methods employ the thermal density-matrix fitting for which a small movement penalty, passed via the option `move_pen` helps stabilizing the convergence. Details on the implementation of the thermal density matrix fitting are given in Appendix A.

The `Lattice` class. At initialization time, an object belonging to the `Lattice` class will require the inputs listed in Table 4. We also list the corresponding physical quantity introduced in prior sections, when applicable. The object `ek_list` is a list of one-body Hamiltonian corresponding to the Fourier transformed superlattice hoppings t^K, as defined in Eq. (9). One can optionally pass an array `wk_list` which contains the multiplicity of each element of the `ek_list` to enable density of state calculations or restricting to the irreducible part of the Brillouin zone.

The two remaining optional arguments control the parallel layer: `use_mpi` switches the distribution of the k-points off, and `comm` allows one to pass an explicit communicator, e.g. a sub-communicator nesting the k-point splitting inside another level of parallelism. Left to their defaults, the code uses `MPI.COMM_WORLD` when mpi4py is available and a single-rank stand-in otherwise, so that a serial run needs no change. The sums over k are distributed over the MPI ranks, each rank owning a contiguous block of k-points of equal size up to one, and closed by an *allreduce* so that all ranks share the same result.

Argument	type	Corresponding quantity
`ek_list`	`np.ndarray`	$\{t^K\}_K$
Optional argument	**type**	**Corresponding quantity**
`wk_list`	`np.ndarray`	
`verbose`	`int`	
`use_mpi`	`bool`	
`comm`	`MPI.Comm`	

Table 4: Inputs used to initialize a `Lattice` object.

4.2 Solvers: the `gemSolver` class, `SimpleED` and generic template

Solving the interacting embedding Hamiltonian of Eq. (15) is the time bottleneck of the method. GEM therefore keeps the solution of the embedding problem behind a small, explicit interface, so that the self-consistency machinery is independent of the strategy used to solve it. A solver has to be passed to each `Fragment` object when the latter is initialized, and the `Fragment` verifies that it is an instance of `gemSolver`.

The `gemSolver` class. Every solver inherits from `gemSolver`, whose constructor stores the two attributes shared by all solvers: 1) `type`, a label used in the printouts that specify the name of the solver, and 2) `solver_params`, a dictionary of solver-specific parameters. A subclass must implement four methods, which are the only pieces that the rest of the code requires:

- `build_Hemb(D, eloc, Lambdac, Utensor)`, which constructs the embedding Hamiltonian from the four tensors that define it;
- `solve_Hemb(T, verbose)`, which solves the embedding Hamiltonian defined before at a temperature `T`. This method also sets the attributes `gs_ene`, the ground-state energy, and `Zpart`, the partition function normalized so that it equals one at zero temperature;
- `calc_density_matrix()`, which calculates and returns the one-body density matrix $\langle c_i^\dagger c_j \rangle$ of the impurity and bath degrees of freedom;

- `compute_E2loc()`, which calculates and returns the local two-body energy.

Two further methods, `compute_E1loc(nimp)` and `calc_double_occ()`, are optional and only needed by the routines that evaluate the corresponding observables (the local one-body energy and the double occupation). Everything else is left to the solver. In particular, the parameters controlling the algorithm are never passed by the `Fragment`: they are fixed once, when the solver is constructed, and read from `solver_params`, which makes the interface independent of the solver in use.

The `SimpleED` solver. GEM ships with an internal exact-diagonalization solver: `SimpleED`, which implements LAPACK based full diagonalization and ARPACK, both via SciPy [32]. It provides the possibility of using symmetry sectors, with the total number of electrons and the total magnetization as good quantum numbers, activated at initialization by passing respectively `use_Ntot=True` and `use_Sz=True`. The user can also fix the symmetry sector by passing the desired list of eigenvalues through the options `N_sector` and `Sz_sector`; leaving them to `None` includes all sectors, which is the correct choice at finite temperature. The `solver_params` dictionary collects the parameters of the diagonalization and of the sampling of the thermal states: the number of eigenpairs `solver_params['num_eig']`, which defaults to the ground state alone at $T = 0$ and to the full spectrum at $T > 0$; the dimension `solver_params['dense_cutoff']` above which a sector is diagonalized with ARPACK rather than fully; the ARPACK options `solver_params['which']` and `solver_params['tol']`, controlling respectively the type of method used and the tolerance; the smallest Boltzmann weight `solver_params['bw_cutoff']` retained in the partition function.

`SimpleED` also implements a matrix-free version of ARPACK meant for calculations with large Hilbert spaces that do not allow storing the full sparse matrices. The matrix-free version can be activated using `solver_params['matrix_free'] = True`. The symmetry sectors are independent of one another, and they are distributed over the ranks of the MPI communicator, passed through the `comm` option and defaulting to `MPI.COMM_WORLD` (the distribution is switched off by `solver_params['use_mpi'] = False`). The sectors differ widely in size, so the assignment is not uniform as for the k-points, but a greedy longest-processing-time-first packing: the sectors are sorted by decreasing estimated cost $\dim^p$ and each is handed to the least loaded rank, with the exponent p set by `solver_params['mpi_weight_exp']`, defaulting to 3 for dense diagonalization and 1 in matrix-free mode, where the cost is a number of matrix-vector products. Each rank then builds the basis and the operators only for the sectors it owns, so that memory scales down with the number of ranks as well, while the partition function, the ground-state energy and the thermal expectation values are obtained by reduction over the ranks and are identical everywhere. The routines involving such reductions are therefore collective and are called by every rank.

Adding a new solver. The file `solver_template.py` contains a commented skeleton that can be copied to interface an external solver: it shows how to inherit from `gemSolver`, marks which arguments are passed by the `Fragment` and which are solver-specific, and documents the quantities each method has to return. When a solver other than `SimpleED` is used, the appropriate references should be cited alongside the GEM one.

4.3 Testing suit

The software comes with unit testing and smoke testing. As shown in Section 2, the testing suit is built by default at installation time, unless explicitly turned off. From the building directory, one can execute the full suit of tests by running `make test`

5 Examples

In this section, we present the functionalities of GEM with a set of paradigmatic calculations for strongly correlated systems. The Python scripts to run these examples can be found in the `gem/examples` folder. The second example is the only one using an external library, TRIQS [24], for the construction of the non interacting Hamiltonian. Nonetheless, it is not necessary and it is just meant to show the possibility of connecting our software with external established packages such as TRIQS. The remaining examples are all dependent on only GEM. The example in Sec. 5.1 presents a finite temperature calculation for the single orbital Hubbard model where the total energy, entropy and double occupancy are evaluated. The example in Sec. 5.2 presents a study of the magnetic phase diagram of the square lattice Hubbard model with two inequivalent fragments for the inequivalent sublattice sites. This example determines the critical temperatures and compares the phase diagram with DMFT results in literature. Finally, the example in Sec. 5.3 presents a three-orbitals calculation for a Hubbard-Kanamori interaction on the Bethe lattice, to identify Hund's physics at fractional filling.

5.1 Bethe lattice: thermodynamics

In this example, we show how to perform finite temperature calculations for the single-orbital Hubbard model on the Bethe lattice. We choose the half-bandwidth (D) of the semi-circular density of state as the unit of energy and we show how to compute the total energy per site and the entropy per site at half-filling for $U/D = 2.0$. The script performing this calculation can be found in `gem/examples/E1/Bethe_1orb.py`. We start from importing the relevant packages:

```
import numpy as np
from gem.fragment import Fragment
from gem.lattice import Lattice
from gem.solvers.simple_ed import SimpleED
```

Next, we define the parameters of the model and the self-consistency. The model has one band and two spins giving `nimp = 2`, with three ghosts per physical orbital $B = 3$. We choose $U/D = 2.0$ and, since the Bethe lattice density of states is particle-hole symmetric, we enforce half-filling by fixing $\mu = U/2$. We choose a set of temperatures between $T/D = 0.001$ and $T/D = 1.0$ on a log grid, in addition to $T = 0$ as the first temperature:

```
# Parameters that determine the physical dimentions
# 1 orbital with 2 spins, B=3 bath per orbital, total 8
B, nimp = 3, 2
nbath = nimp*B
ntot = nimp+nbath

# Physical Parameters
U, mu = 2.0, 1.0
Tlist = np.hstack((np.array([0]), np.logspace(np.log10(1e-3), np.log10(1), 31)))

# Self-consistency Parameters
itmax, tol, Tsmearing = 100, 1e-5, 1e-3
```

The last line is additional parameters: we perform a maximum of `itmax` iterations, unless we reach the convergence criterion set by `tol`. Moreover, we use a small smearing for the temperature `Tsmearing` so that the Fermi function of the *quasiparticle* problem does not have a jump at zero temperature.

We then define the kinetic part of the Hamiltonian and initialize a `Lattice` object containing this information:

```
# Non-interacting density of states and lattice object for the Bethe lattice
```

```
e_list = np.linspace(-1, 1, 5001)
wks = np.sqrt(1 - e_list**2)
wks /= np.sum(wks)
eks = e_list[:, None, None] * np.eye(2, dtype=np.complex128)

lattice = Lattice(eks, wk_list=wks)
```

and the local Hamiltonian together with a `SimpleED` solver and a `Fragment` object:

```
# Local Hamiltonian
eloc = np.zeros((nimp,nimp))
# Interaction tensor of the embedded space
Utensor = np.zeros((nimp, nimp, nimp, nimp))
Utensor[0,0,1,1], Utensor[1,1,0,0] = U, U

# SimpleED solver initialization
edsolver = SimpleED(ntot, use_Ntot=True, use_Sz=True, N_sector=None, Sz_sector=None,
                    dtype=np.float64)

# Fragment initialization
Lambda0 = None; R0 = None
fragment = Fragment(nimp, nbath, eloc, Utensor, edsolver, Lambda=Lambda0, R=R0,
                    verbose=2)
```

We define a function to check for convergence of $\mathcal{R}$ and Λ matrices. Due to the gauge freedom of these matrices, we check convergence for gauge invariant quantities:

```
def check_convergence(R_new, L_new, R_old, L_old):
    # Only 1 spin and gauge invariant difference
    L_eval_new, UL_new = np.linalg.eigh(L_new[::2,::2])
    L_eval_old, UL_old = np.linalg.eigh(L_old[::2,::2])
    R_eval_old, R_eval_new = UL_old @ R_old[::2,::2], UL_new @ R_new[::2,::2]
    diff_R = np.abs(np.abs(R_eval_old) - np.abs(R_eval_new)).max()
    diff_Lambda = np.abs(L_eval_new - L_eval_old).max()
    diff = max(diff_R, diff_Lambda)
    return diff
```

And finally, we setup a self-consistency cycle storing the total energy, entropy and double occupancies:

```
Elist, Slist, docclist = [], [], []

for T in Tlist:
    print('---------------------------------------')
    print(f'GEM loop started with U={U} and T={T}')

    # Self consistency loop
    for it in range(itmax):
        print(f'----- ghost-RISB iteration {it} / {itmax} -----')
        lattice.solve_qp([fragment], T=T, Tsmearing=Tsmearing) # Step 1: Solve H_qp
        fragment.update_hybridization(T=T, use_Sz=True) # Step 2: Update Delta
        fragment.solve_impurity(mu, T=T) # Step 3: Solve H_emb
        Lambda_old, R_old = fragment.Lambda.copy(), fragment.R.copy() # Save old Sigma
        fragment.update_self_energy(T=T, use_Sz=True) # Step 4: Update Sigma
        Lambda_new, R_new = fragment.Lambda, fragment.R # Save new Sigma

        diff = check_convergence(R_new, Lambda_new, R_old, Lambda_old)
        print(f'iteration: {it}  diff={diff}')

        if (diff < tol and it > 2) or it == itmax - 1:
            print(f'----- Exiting loop with diff={diff} after '
                  f'{it} iterations (max={itmax}) -----'); break

    docc = fragment.E2loc/U
```

```
    docclist.append(docc)
    ekin = lattice.compute_ekin([fragment], T=T, Tsmearing=Tsmearing)
    eimp = fragment.compute_energy()
    etot = (eimp+ekin).real
    # Free-energy functional
    L = lattice.compute_functional([fragment], T=T, Tsmearing=Tsmearing).real
    S = (etot-L)/T if T > 0 else 0.0

    Elist.append(etot); Slist.append(S)

    print('----------------------------------------')
    print(f'GEM loop ended with U={U} and T={T}')
    print(f'returning docc={docc}, Etot={etot} and S={S}')
    print('----------------------------------------')

np.savetxt('Tlist.dat', Tlist); np.savetxt('Elist.dat', Elist)
np.savetxt('Slist.dat', Slist); np.savetxt('docclist.dat', docclist)
```

In Fig. 2, we report the behavior of the total energy per site (E), entropy per site (S) and double occupancies ($\langle n_\uparrow n_\downarrow \rangle$) as a function of temperature (T). The $T = 0$ values (red dotted line) are compatible with the finite temperature ones (blue line). This is in agreement with Ref. [5] where a comparison with DMFT was also given.

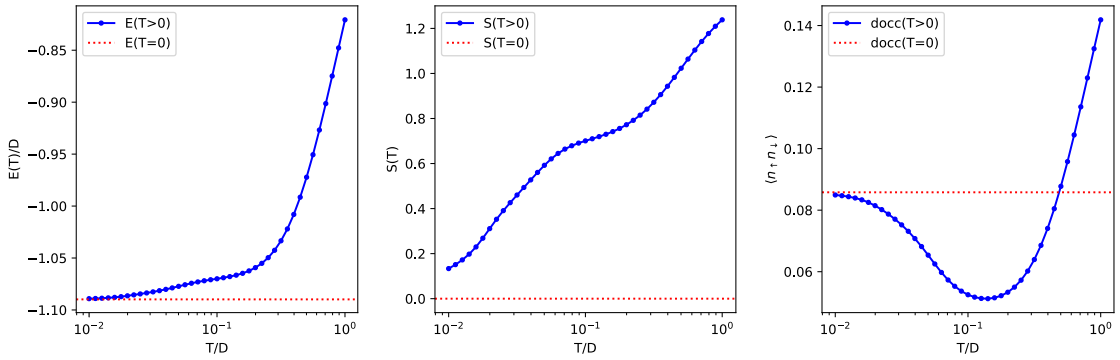

Figure 2: Temperature evolution of the total energy per site E (left), entropy per site S (center) and double occupancy (right) for the Hubbard model on the Bethe lattice with $U/D = 2.0$ using $B = 3$.

5.2 Square lattice: magnetic phase diagram

In this example, we show how to perform finite-temperature calculations for computing the magnetic phase diagram of the single-orbital Hubbard model on the square lattice at half-filling. We use two inequivalent fragments as an example of a multi-fragment calculation. We choose the half-bandwidth ($D = 4t$) as the unit of energy, therefore $t = 0.25$, and compute the antiferromagnetic order parameter along the z-axis, that is $\langle n_\uparrow - n_\downarrow \rangle$, as a function of temperature. We then extract the critical temperature (T_c) and compare with DMFT data obtained from Ref. [42]. We make use of TRIQS [24] to generate the dispersion `ek_list` from a tight-binding object, which is then passed to the `Lattice` object. The script performing this calculation can be found in `gem/examples/E2/Square_1orb_2frag_phase.py`.

We start again from importing the relevant packages:

```
import numpy as np
from gem.fragment import Fragment
from gem.lattice import Lattice
from gem.solvers.simple_ed import SimpleED
```

```
import h5py
```

We then define the parameters of the model and the self-consistency. We present a script performing a calculation for a single U. We impose half-filling by setting the chemical potential to be $\mu = U/2$. The temperatures are linearly spaced between $T/D = 0$ and $T/D = 0.1$. We set the hopping to $t = 0.25$ and include a small magnetic field `bfield` that helps stabilizing the antiferromagnetic solution at low temperatures for the first few iterations. Moreover, we use a small linear mixing `mix` for Λ and $\mathcal{R}$ and, as shown in Appendix A, we use a small movement penalty `move_pen` for the thermal density matrix fitting that helps stabilizing convergence.

```
# Parameters that determine the physical dimensions
# 1 orbital with 2 spins, B=3 bath per orbital, total 8
nimp, B = 2, 3
nbath = nimp * B
ntot = nimp + nbath

# Physical Parameters
U, mu, t, bfield = 0.8, 0.4, 0.25, 1e-2
T_list = np.linspace(0.0, 0.10, 51)

# Self-consistency Parameters
itmax, tol, mix, move_pen, Tsmearing = 200, 1e-3, 0.05, 1e-8, 1e-3
# Output file
h5_file = f'data_Square_1orb_2frag_B{B}_phase.h5'
```

We then make use the `TBLattice` object from TRIQS to generate the dispersion and initialize the `Lattice` object:

```
from triqs.lattice.tight_binding import TBLattice

# Two-site (Neel) unit cell of the square lattice: A at (0,0), B at (1,0),
# primitive vectors along the diagonals so that the two sublattices alternate.
a1, a2 = np.array([1.0,  1.0, 0.0]), np.array([1.0, -1.0, 0.0])
H_t = TBLattice(
    units=[a1, a2],
    hoppings={( 0,  0): [[0, -t], [-t, 0]],
              (-1, -1): [[0, -t], [ 0, 0]], ( 1,  1): [[0,  0], [-t, 0]],
              ( 0, -1): [[0, -t], [ 0, 0]], ( 0,  1): [[0,  0], [-t, 0]],
              (-1,  0): [[0, -t], [ 0, 0]], ( 1,  0): [[0,  0], [-t, 0]]},
    orbital_positions=[(0, 0, 0), (1, 0, 0)], orbital_names=['A', 'B'],
)

# Build the k-grid
Nk = 200
kmesh = H_t.get_kmesh((Nk, Nk, 1))
kpts_cart = np.array(list(kmesh.values()))
recip = np.array(kmesh.bz.units)              # rows: reciprocal lattice vectors
kpts = kpts_cart @ np.linalg.inv(recip)

# List of H(k) matrices on the square-lattice BZ, one spin block per sublattice
eks = np.array([np.kron(H_t.fourier(k), np.eye(2)) for k in kpts])
lattice = Lattice(eks)
```

We then define the local Hamiltonian and initialize solvers and fragment objects for each sublattice:

```
# Local Hamiltonian
eloc = np.zeros( (nimp,nimp) )
# Interaction tensor of the embedded space
Utensor = np.zeros((nimp, nimp, nimp, nimp))
Utensor[0, 0, 1, 1], Utensor[1, 1, 0, 0] = U, U

# Let the fragment decide a starting value
```

```
Lambda_A, R_A = None, None;   Lambda_B, R_B = None, None
Lambda_c_A, D_A = None, None; Lambda_c_B, D_B = None, None

#Initalize solvers and fragments for the two sublattices
edsolverA = SimpleED(ntot, use_Ntot=True, use_Sz=True, dtype=np.complex128)
edsolverB = SimpleED(ntot, use_Ntot=True, use_Sz=True, dtype=np.complex128)
fragmentA = Fragment(nimp, nbath, eloc, Utensor, edsolverA, verbose=0,
                     Lambda=Lambda_A, R=R_A, Lambda_c=Lambda_c_A, D=D_A)
fragmentB = Fragment(nimp, nbath, eloc, Utensor, edsolverB, verbose=0,
                     Lambda=Lambda_B, R=R_B, Lambda_c=Lambda_c_B, D=D_B)
```

We then create the output file and an array to store the magnetizations and other quantities:

```
# create the file if not present; each U group is appended as it completes
with h5py.File(h5_file, 'a') as _:  # 'a' append, 'w' write and delete previous
    pass

# mag_grid[:,iT] = [ m_A, m_B ] order parameter on sublattice A and B
mag_grid = np.zeros( (2,len(T_list)) )
magA = 0.0; magB = 0.0
```

Finally, we write the self-consistency cycle checking the convergence on both the Λ and $\mathcal{R}$ matrices, as well as on the magnetization on each sublattice:

```
for iT, T in enumerate(T_list):
    print('------------------------------------------------------------')
    print(f'GEM loop started with U={U} and T={T}')

    # Self consistency loop
    for it in range(itmax):
        print(f"----- ghost-RISB iteration {it} / {itmax} -----")

        lattice.solve_qp([fragmentA, fragmentB], T=T, Tsmearing=Tsmearing)

        fragmentA.update_hybridization(T=T, use_Sz=True, move_pen=move_pen)
        fragmentB.update_hybridization(T=T, use_Sz=True, move_pen=move_pen)

        # seed AFM only at lowest T for the first few iterations
        if iT == 0 and it < 3:
            fragmentA.eloc = eloc + bfield * np.diag([-1, 1])
            fragmentB.eloc = eloc - bfield * np.diag([-1, 1])
        else:
            fragmentA.eloc = eloc.copy()
            fragmentB.eloc = eloc.copy()

        fragmentA.solve_impurity(mu, T=T)
        fragmentB.solve_impurity(mu, T=T)

        Lambda_old_A = fragmentA.Lambda.copy(); R_old_A = fragmentA.R.copy()
        Lambda_old_B = fragmentB.Lambda.copy(); R_old_B = fragmentB.R.copy()

        fragmentA.update_self_energy(T=T, use_Sz=True, move_pen=move_pen)
        fragmentB.update_self_energy(T=T, use_Sz=True, move_pen=move_pen)

        diff_LR = max(
            np.abs(fragmentA.Lambda - Lambda_old_A).max(),
            np.abs(np.abs(fragmentA.R) - np.abs(R_old_A)).max(),
            np.abs(fragmentB.Lambda - Lambda_old_B).max(),
            np.abs(np.abs(fragmentB.R) - np.abs(R_old_B)).max(),
        )

        # Mixing of Lambda and R
        fragmentA.Lambda = (1 - mix) * fragmentA.Lambda + mix * Lambda_old_A
        fragmentA.R = (1 - mix) * fragmentA.R + mix * R_old_A
```

```
        fragmentB.Lambda = (1 - mix) * fragmentB.Lambda + mix * Lambda_old_B
        fragmentB.R = (1 - mix) * fragmentB.R + mix * R_old_B

        magA_old, magB_old = magA, magB
        magA = (fragmentA.denMat[0, 0].real - fragmentA.denMat[1, 1].real)
        magB = (fragmentB.denMat[0, 0].real - fragmentB.denMat[1, 1].real)

        diff_mA = abs(magA - magA_old)
        diff_mB = abs(magB - magB_old)
        diff = max(diff_LR, 10 * diff_mA, 10 * diff_mB)

        print(f'iteration: {it}  diff={diff:.2e} '
                f'diff_mA={diff_mA:.4f} diff_mB={diff_mB:.4f} '
                f'magA={magA:.4f} magB={magB:.4f} '
                f'nfillA={fragmentA.nfill:.4f} nfillB={fragmentB.nfill:.4f} ')

        if (diff < tol and it > 1) or it == itmax - 1:
            print(f"--- Exiting after {it}/{itmax} - diff={diff:.2e}  ---")
            break

    magA = (fragmentA.denMat[0,0].real - fragmentA.denMat[1,1].real)
    magB = (fragmentB.denMat[0,0].real - fragmentB.denMat[1,1].real)
    mag_grid[0,iT] = magA; mag_grid[1,iT] = magB


    print('-------------------------------------------------------')
    print(f'GEM loop ended with U={U} and T={T}')
    print(f'returning m_A={mag_grid[0,iT]} and m_B={mag_grid[1,iT]}')
    print('-------------------------------------------------------')
```

After convergence we save the data in the output file:

```
# save to output file
with h5py.File(h5_file, 'a') as h5f:
    grp_name = f'U{U:.2f}_B{B}'
    if grp_name in h5f:
        del h5f[grp_name]
    grp = h5f.create_group(grp_name)
    grp.create_dataset('T_list',   data=T_list)
    grp.create_dataset('mag_grid', data=mag_grid)
print(f'Written U={U:.2f} to {h5_file}')
```

In Fig. 3 (left), we plot the antiferromagnetic order parameter defined as the magnetization per site ($m_z = \langle n_{iA\uparrow} - n_{iA\downarrow} \rangle = \langle n_{iB\downarrow} - n_{iB\uparrow} \rangle$).

We use $B = 3$ and see that it correctly captures the BCS-BEC crossover with a maximum critical temperature reached for $U/D \approx 2$. In Figure 3(right), we compare the phase boundaries obtained by GEM with the DMFT phase boundaries obtained from Ref. [42].

5.3 Multi-orbital Bethe lattice: Hund's physics

In this example, we show how to perform zero temperature calculations for a multi-orbital system with Hubbard-Kanamori rotationally-invariant interaction [43]. We focus on 3-orbitals on the Bethe lattice and show how the quasiparticle weight evolve with interaction and the increase of the critical U for values of J/U close to $1/3$. We choose the half-bandwidth (D) of the semi-circular density of state as the unit of energy and we compare with GA and ghost-GA calculations found in literature, in particular Refs. [44, 45]. The script performing this calculation can be found in `gem/examples/E3/Bethe_3o.py`.

We start from importing the relevant packages:

```
import numpy as np
from gem.fragment import Fragment
```

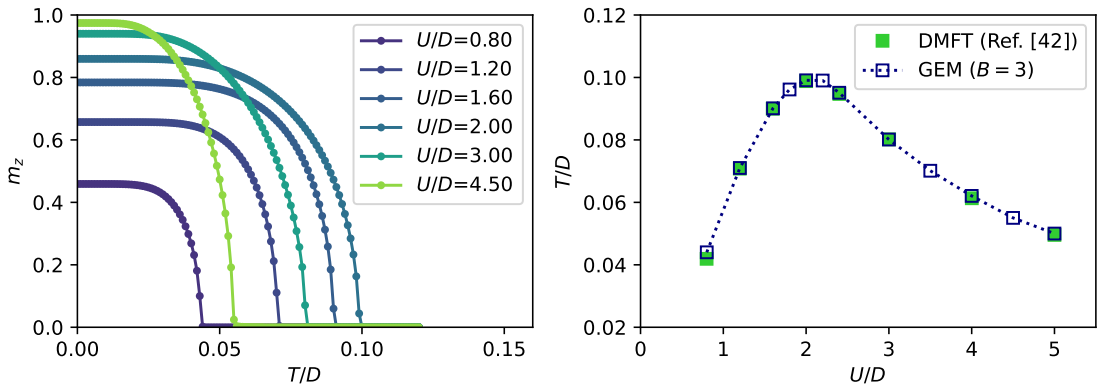

Figure 3: Antiferromagnetic order parameter (left) as a function of temperature for different values of U/D obtained with GEM using $B = 3$, and temperature versus interaction antiferromagnetic phase boundaries (right) for the square lattice Hubbard model compared with DMFT from Ref. [42].

```
from gem.lattice import Lattice
from gem.solvers.simple_ed import SimpleED
from gem.utilities import U_matrix_kanamori
```

Note that we imported the `U_matrix_kanamori` function implemented in the GEM utilities module. Next, we define the parameters of the model and of self-consistency:

```
n_orb, B = 3, 3 # B will be 1 or 3 in this example
nimp = 2 * n_orb
nbath = nimp * B
ntot = nimp + nbath

# Physical Parameters
U_list, JoverU_list = np.linspace(0.1,10.0,100), np.array([0.00, 0.10, 0.20, 0.30])
T, n_target = 0, n_orb-1

# Self-consistency Parameters
itmax, tol, mix, ntol, Tsmearing = 200, 1e-4, 0.2, 1e-4, 1e-3
```

In this calculation, `tol` is the tolerance on the change of Λ and R, that serves as the convergence criterion for the self-consistency. The variable `ntol` is the tolerance on the impurity filling.

We then define the kinetic part of the Hamiltonian and initialize a `Lattice` object containing this information:

```
# Non-interacting density of states and lattice object
e_list = np.linspace(-1, 1, 1001)
wks = np.sqrt(1 - e_list**2)
wks /= np.sum(wks)
eks = e_list[:, None, None] * np.eye(nimp, dtype=np.complex128)
lattice = Lattice(eks, wk_list=wks)
```

and a function to check the convergence:

```
def check_convergence(R_new, L_new, R_old, L_old):
    # Only 1 spin and gauge invariant difference
    L_eval_new, UL_new = np.linalg.eigh(L_new[::2*n_orb, ::2*n_orb])
    L_eval_old, UL_old = np.linalg.eigh(L_old[::2*n_orb, ::2*n_orb])
    diff_R = ( np.abs(np.abs(UL_old @ R_old[::2*n_orb, ::2*n_orb])
         - np.abs(UL_new @ R_new[::2*n_orb, ::2*n_orb])) ).max()
    diff_Lambda = np.abs(L_eval_new - L_eval_old).max()
    diff = max(diff_R, diff_Lambda)
    return diff
```

We initialize arrays to store the information for the future plots and then built the cycle similarly to what was shown in the previous examples:

```
Zgrid = np.zeros((len(JoverU_list), len(U_list)))
ngrid = np.zeros((len(JoverU_list), len(U_list)))

for iJ, JoverU in enumerate(JoverU_list):
    # warm-start: carry the converged Lambda/R and mu from one U to the next
    Lambda0, R0, mu = None, None, 0
    for iU, U in enumerate(U_list):
        J = JoverU * U
        print('------------------------------------------------------------')
        print(f'GEM loop started with U={U:.2f} and J={J:.2f} (J/U={JoverU:.2f})')

        # Local Hamiltonian: the level position is set by mu, fitted to n_target
        eloc = np.zeros((nimp, nimp))
        # Interaction tensor of the embedded space
        Utensor = U_matrix_kanamori(n_orb, U, J)

        # Fixed sectors for T=0 calculations
        edsolver = SimpleED(ntot, use_Ntot=True, use_Sz=True,
                            N_sector=ntot//2, Sz_sector=0, dtype=np.float64)

        # Fragment initialization
        fragment = Fragment(nimp, nbath, eloc, Utensor, edsolver,
                            Lambda=Lambda0, R=R0, verbose=0)

        # Self consistency loop
        for it in range(itmax):
            print(f"----- ghost-RISB iteration {it} / {itmax} -----")
            lattice.solve_qp([fragment], T=T, Tsmearing=Tsmearing)
            fragment.update_hybridization(T=T, use_Sz=True)
            fragment.solve_impurity(mu, T=T)

            #Fit density to n_target
            if abs(fragment.nfill.real - n_target) > ntol:
                mu = lattice.fit_mu(n_target, [fragment], T=T, mu_old=mu,
                                    mode='imp', ntol=1e-5)
                fragment.solve_impurity(mu, T=T)
            nfill=fragment.nfill.real

            Lambda_old = fragment.Lambda.copy(); R_old = fragment.R.copy()
            fragment.update_self_energy(T=T, use_Sz=True)
            # stay in the paramagnetic, orbitally degenerate solution
            fragment.impose_orbital_symmetry()
            fragment.impose_spin_SU2_symmetry()
            Lambda_new = fragment.Lambda; R_new = fragment.R

            diff = check_convergence(R_new, Lambda_new, R_old, Lambda_old)
            print(f"it:{it}  diff={diff:.3e}  mu={mu:.4f}  nfill={nfill:.4f}")

            fragment.Lambda = (1 - mix) * Lambda_new + mix * Lambda_old
            fragment.R = (1 - mix) * R_new + mix * R_old

            if (diff < tol and it > 2) or it == itmax - 1:
                print(f"-- Exiting with diff={diff:.3e} after {it} iterations")
                break

        Z = fragment.compute_Z()
        Zgrid[iJ, iU], ngrid[iJ, iU] = Z.real[0, 0], fragment.nfill.real

        # warm start for the next U
```

```
        Lambda0, R0 = fragment.Lambda.copy(), fragment.R.copy()

        print('------------------------------------------------------------')
        print(f'GEM loop ended with U={U} and J={J}')
        print(f'returning Z={np.diag(Z.real)}, mu={mu} and n={fragment.nfill.real}')
        print('------------------------------------------------------------')

np.savetxt('hund_Ulist.dat', U_list); np.savetxt('hund_JoverU.dat', JoverU_list)
np.savetxt('hund_Zgrid.dat', Zgrid);  np.savetxt('hund_ngrid.dat', ngrid)
```

In Fig. 4, we show the results of the calculations for $J/U = 0.0, 0.1, 0.2$ and 0.3, where we plot the quasiparticle weight Z as a function of U/D for $B = 1$ (left panel) and $B = 3$ (right panel). The results can be compared with Ref. [44] for $B = 1$ and with Ref. [45] for $B = 3$. When Hund's physics becomes important around $U/J \approx 1/3$ in this system, $B = 3$ shows a much larger renormalization of the quasiparticle weight and is in much better agreement with DMFT, as shown in Ref. [45].

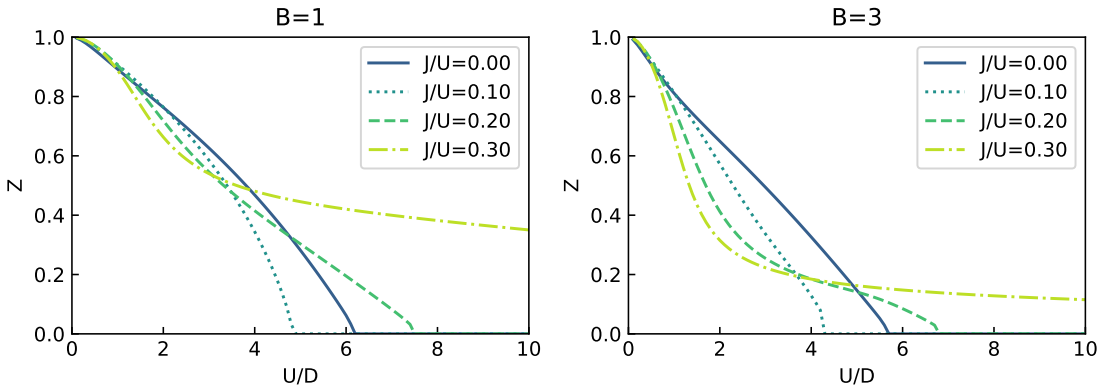

Figure 4: Quasiparticle weight Z for a three-orbital model with local Hubbard-Kanamori interaction on the Bethe lattice at filling n=2 as a function of U/D and for $J/D = 0.0, 0.1, 0.2$ and 0.3 using GEM, for $B = 1$ (left panel) and $B = 3$ (right panel).

6 Outlook

The modular structure of GEM provides a natural framework for extending its capabilities and interfacing it with other computational methods and software packages, and several directions for future development are being explored.

One direction concerns the integration of additional impurity solvers. In particular, interfaces to ground-state solvers based on density-matrix renormalization group (DMRG) and matrix product states (MPS) methods, coupled-cluster approaches, and other approximate ground-state solvers [39] could considerably broaden the range of systems and regimes accessible within GEM.

Another extension is the inclusion of retarded interactions, which would allow GEM to address a wider class of effective low-energy models with frequency-dependent interactions that have been shown to be a promising route for this method [46].

Finally, interfacing GEM with *ab initio* electronic-structure codes will provide a route towards material-specific calculations. At the moment, an interface with TRIQS/MODEST [47] is under development, enabling DFT+ghost-GA calculations, which have already proven to be a

cheap and fast alternative to DFT+DMFT [48]. Such an interface will facilitate the construction of realistic low-energy models from first-principles calculations and their subsequent treatment within the ghost embedding framework. Extending GEM toward charge self-consistency is also appealing for material applications, such as systematic studies on competing phases across structural or stoichiometric phase spaces.

7 Conclusions

In this work, we have presented the Ghost Embedding Method package (GEM), an open-source implementation of the ghost-Gutzwiller approximation in its quantum-embedding formulation, for both zero- and finite-temperature calculations. GEM provides a practical implementation of the ghost-embedding functional and its self-consistency conditions, formulated in terms of one-body density matrices rather than frequency-dependent Green's functions as in DMFT. The number of auxiliary bath modes provides a systematic way of enlarging the variational space, connecting the Gutzwiller approximation at small bath size to DMFT in the infinite bath limit. The software is organized around separate lattice, fragment, and impurity-solver components. This modular structure separates the ghost-embedding self-consistency from the solution of the interacting embedding problem and facilitates the implementation and testing of alternative impurity solvers. GEM includes an exact-diagonalization solver and provides a common interface through which external solvers can be incorporated. We have illustrated the use of GEM through three representative applications. These examples demonstrate finite-temperature calculations of thermodynamic observables for the single-orbital Hubbard model, the determination of a finite-temperature magnetic phase diagram using multiple correlated fragments, and a zero-temperature calculation of a multi-orbital Hubbard-Kanamori model. Together, they illustrate the applicability of the same framework to thermodynamic properties, symmetry-broken phases, and multi-orbital correlated systems. By providing the ghost-embedding approach as a documented and reusable software package, GEM makes this intermediate level of approximation between inexpensive static Gutzwiller methods and fully dynamical DMFT calculations accessible for model calculations, benchmarking, and methodological developments. We expect its modular solver interface and compatibility with the broader scientific Python ecosystem to provide a basis for further developments and applications of ghost-embedding methods.

Acknowledgments

We acknowledge helpful discussions with N. Hogan, G. Bellomia, C. Mejuto-Zaera, E. Zavatti, P. Filice, F. Kugler, A.M. Tagliente and M. Capone at various stages of development.

Funding information The Flatiron Institute is a division of the Simons Foundation. N.L. gratefully acknowledges funding from the National Science Foundation under Award No. DMR-2532771 and from the Simons Foundation (Grant No. 00024037). Work by Y.Y. was supported by the U.S. Department of Energy (DOE), Office of Science, Basic Energy Sciences, Materials Science and Engineering Division, including the grant of computer time at the National Energy Research Scientific Computing Center (NERSC) in Berkeley, California. This part of research was performed at the Ames National Laboratory, which is operated for the U.S. DOE by Iowa State University under Contract No. DE-AC02-07CH11358. T.-H. L. gratefully acknowledges funding from the National Science and Technology Council (NSTC) of Taiwan under Grant Nos. NSTC 112-2112-M-194-007-MY3 and 115-2112-M-194-005, as well as support from the

National Center for Theoretical Sciences (NCTS) in Taiwan.

A Density-matrix fitting

The self-consistency conditions of the ghost-embedding functional Eq. (13) are given by the saddle-point equations Eqs. (17-20). These are expressed as matching conditions of one-body density matrices belonging to the three auxiliary problems: the *quasiparticle* problem with Hamiltonian given by Eq. (14), the *interacting embedding* problem with Hamiltonian given by Eq. (15) and the *non-interacting embedding* problem with Hamiltonian given by Eq. (16).

At zero temperature, the ghost-Gutzwiller stationarity conditions can be solved in closed form, as shown in Ref. [5]. The auxiliary state is a Slater determinant, and the mixed expectation values entering the update of the variational matrices are algebraic functions of the quasiparticle density matrix, involving only the matrix square root $[\Delta(1-\Delta)]^{1/2}$, where $\Delta_{\alpha\beta} = \langle f_\alpha^\dagger f_\beta\rangle = \langle b_\beta b_\alpha^\dagger\rangle$.

At finite temperature this algebraic structure is lost. GEM therefore imposes the finite-temperature conditions directly, as a pair of nonlinear least-squares problems in which a *thermal* one-body density matrix is fitted to a set of target blocks. This appendix documents that procedure, implemented in `python/gem/delta_fit.py` and driven by `Fragment.update_self_energy` and `Fragment.update_hybridization`.

We consider a single fragment with ν physical (impurity) orbitals and $B\nu$ auxiliary (ghost) levels, where B is the number of ghost levels per physical orbital.

The `Fragment.update_self_energy` routine. The fitting of Λ_i and $\mathcal{R}_i$ is performed to satisfy the saddle point condition of Λ_i^c, Eq. (20), and of $\mathcal{D}_i$, Eq. (19). The fit of the one-body density matrix is performed by the `Fragment.update_self_energy` routine. Both conditions involve the one-body density matrix of the non-interacting embedding problem defined in Eq. (16). Denoting with $H_i^{0,\mathrm{emb}}$ the $2B\nu\times 2B\nu$ single-particle matrix representation of the quadratic embedding Hamiltonian $\hat{H}_i^{0,\mathrm{emb}}$, the corresponding one-body density matrix at temperature $T=1/\beta$ is

$$\mathcal{F}_i(\Lambda_i,\mathcal{R}_i;\Lambda_i^c,\mathcal{D}_i) \;=\; \Big[\, f_\beta\big(H_i^{0,\mathrm{emb}}\big)\Big]^T, \qquad f_\beta(x)=\frac{1}{1+e^{\beta x}}, \tag{A.1}$$

where the matrix function is evaluated by diagonalizing $H_i^{0,\mathrm{emb}}$. the transpose enforces the convention $[\mathcal{F}_i]_{bc} = \langle a_{ib}^\dagger a_{ic}\rangle$ (where a can either be an f or a b electron) used throughout GEM. We denote by $\mathcal{F}_i^{11}$, $\mathcal{F}_i^{12}$ and $\mathcal{F}_i^{22}$ its $B\nu\times B\nu$ blocks, where 1 labels the f fermions (quasiparticle) sector, in which Λ_i acts, and 2 the b fermions (embedding-bath) sector, in which $-\Lambda_i^c$ acts. All the dependence of the saddle-point conditions Eqs. (19, 20) on Λ_i and $\mathcal{R}_i$ is contained in Eq. (A.1). The left-hand sides are the corresponding blocks $\langle b_{ia}^\dagger b_{ib}\rangle_{\mathrm{emb}}$ and $\langle c_{i\alpha}^\dagger b_{ib}\rangle_{\mathrm{emb}}$ of the one-body density matrix of the *interacting* embedding problem Eq. (15), which are computed when calling `Fragment.solve_impurity`. They are held fixed at the values returned by the impurity solver during the fit.

The routine therefore minimizes the cost function

$$\chi^2_{i,\Sigma}(\Lambda_i,\mathcal{R}_i) = \sum_{a\le a'}\Big|\big[\mathcal{F}_i^{22}\big]_{aa'} - \langle b_{ia}^\dagger b_{ia'}\rangle_{\mathrm{emb}}\Big|^2 + \sum_{a,\alpha}\Big|\big[\mathcal{R}_i^T\,\mathcal{F}_i^{12}\big]_{\alpha a} - \langle c_{i\alpha}^\dagger b_{ia}\rangle_{\mathrm{emb}}\Big|^2 + \alpha_{\mathrm{p}}\,\big\|\,\mathsf{y}-\mathsf{y}_0\big\|^2, \tag{A.2}$$

where the first sum runs over the independent components of a Hermitian matrix and y is an array that collects all the independent real parameters of Λ_i and $\mathcal{R}_i$, with y_0 their value at the previous iteration of the self-consistency cycle. Discarding for a moment the last term, the two

residual blocks of Eq. (A.2) contain $(B\nu)^2$ and $2B\nu^2$ independent real equations respectively, exactly as many as the free real parameters of a Hermitian Λ_i and of a complex $\mathcal{R}_i$: the fit formally is a square nonlinear root-finding problem, which is however solved as a least-squares problem for the reasons given below.

The minimization of the distance is carried out with the Gauss-Newton algorithm implemented in `scipy.optimize.least_squares`, using the analytic Jacobian of the residuals. The only non-trivial ingredient of the latter is the derivative of the matrix Fermi function Eq. (A.1), which is not the naive derivative of f_β, but rather its Fréchet derivative [49], evaluated in the spectral (Daleckii–Kreĭn) representation [50, Eq. (2.11) and Cor. 2.9]

$$\delta\mathcal{F}_i = \Big[\, U\big(G \circ U^\dagger\, \delta H_i^{0,\mathrm{emb}}\, U\big)U^\dagger\Big]^T, \qquad G_{mn} = \frac{f_\beta(\varepsilon_m) - f_\beta(\varepsilon_n)}{\varepsilon_m - \varepsilon_n}, \tag{A.3}$$

with $H_i^{0,\mathrm{emb}} = U\,\mathrm{diag}(\varepsilon)\,U^\dagger$, $G_{mn} \to f'_\beta(\varepsilon_m)$ in the degenerate limit $\varepsilon_m \to \varepsilon_n$ and $\circ$ the Hadamard (element-wise) product. The cost of the fit is dominated by the diagonalization of $\mathcal{H}_i^{0,\mathrm{emb}}$ that is a quadratic Hamiltonian and is therefore negligible compared to a single solution of the interacting embedding problem.

The last term of Eq. (A.2) is a proximal penalty that keeps the solution close to the previous iteration. It is needed, because the residuals are invariant under the gauge transformation $\Lambda_i \to u\,\Lambda_i\,u^\dagger$, $\mathcal{R}_i \to u\,\mathcal{R}_i$, with u any unitary transformation in the space of $B\nu \times B\nu$ matrices, which reflects the freedom in the choice of the ghost basis.

A second degeneracy appears at low temperature, where the division of differences in $G_m n$ of Eq. (A.3) are exponentially small for ghost levels far from the chemical potential, and therefore the corresponding directions become numerically flat. The penalty regularizes both, shifting all the singular values of the augmented Jacobian by at least $\sqrt{2\alpha_\mathrm{p}}$, and selects among the gauge-equivalent solutions the one closest to y_0, which keeps the ordering and the phases of the ghost levels continuous along the self-consistency cycle and along parameter scans. It does not bias the converged result: at a fixed point of the cycle, $\mathrm{y} = \mathrm{y}_0$, and the penalty vanishes together with its gradient. In practice, α_p is taken in the range 10^{-8}–10^{-6} (argument `move_pen`, default 10^{-6}) and can be increased or decreased adaptively if a given step fails to converge.

Finally, the fit is performed on a real parametrization of $(\Lambda_i, \mathcal{R}_i)$, obtained by packing into a single real vector y the real upper triangle of Λ_i including the diagonal, its strict imaginary upper triangle, and the real and imaginary parts of $\mathcal{R}_i$ (routines `pack_params` and `unpack_params` in `delta_fit.py`). Hermiticity of Λ_i is thus imposed by construction rather than as a constraint, and the least-squares solver operates on unconstrained real variables. When S_z is a good quantum number, the whole procedure is applied independently to each spin block (`use_Sz=True`), which halves the total number of fitted parameters and reduces the cost of the fitting.

The `Fragment.update_hybridization` routine. The fitting of Λ_i^c and $\mathcal{D}_i$ is performed to satisfy the saddle point condition of Λ_i and $\mathcal{R}_i$, provided by Eqs.(18, 17). They closely mimic the fit of Λ_i and $\mathcal{R}_i$ presented in the previous paragraph, but in this case, the fit of the one-body density matrix is performed by the `Fragment.update_hybridization` routine.

Both conditions involve the one-body density matrix of the non-interacting embedding problem given by Eq. (16), as previously shown in Eq. (A.1), and again all the dependence of the saddle-point conditions on Λ_i^c and $\mathcal{D}_i$, given by Eqs. (17, 18), is contained in Eq. (A.1). The left-hand sides are the corresponding blocks $\langle f_{ia}^\dagger f_{ib}\rangle_\mathrm{qp}$ and $\sum_{j=1}^{\mathcal{N}}\sum_{\beta=1}^{\nu_j}\sum_{b=1}^{B\nu_j}[t^{ij}]_{\alpha\beta}[\mathcal{R}_j^\dagger]_{\beta b}\langle f_{ia}^\dagger f_{jb}\rangle_\mathrm{qp}$ of the one-body density matrix of the *quasiparticle* problem Eq. (14), which are computed when calling `Lattice.solve_qp`, are stored inside each fragment object, and are held fixed during

the fit. For simplicity we will call:

$$\sum_{j=1}^{\mathcal{N}} \sum_{\beta=1}^{\nu_j} \sum_{b=1}^{B\nu_j} [t^{ij}]_{\alpha\beta} [\mathcal{R}_j^\dagger]_{\beta b} \langle f_{ia}^\dagger f_{jb} \rangle_{\mathrm{qp}} = [\Gamma_i^{\mathrm{qp}}]_{a\alpha}. \tag{A.4}$$

The routine therefore minimizes the cost function

$$\chi^2_{i,\Delta}(\Lambda_i^c, \mathcal{D}_i) = \sum_{a \le a'} \left| [\mathcal{F}_i^{11}]_{aa'} - \langle f_{ia}^\dagger f_{ia'} \rangle_{\mathrm{qp}} \right|^2 + \sum_{a,\alpha} \left| [\mathcal{F}_i^{12} \mathcal{D}_i]_{a\alpha} - [\Gamma_i^{\mathrm{qp}}]_{a\alpha} \right|^2 + \alpha_{\mathrm{p}} \left\| \mathsf{y} - \mathsf{y}_0 \right\|^2, \tag{A.5}$$

where the first sum runs over the independent components of a Hermitian matrix and y is an array that collects all the independent real parameters of Λ_i^c and $\mathcal{D}_i$, with y_0 their value at the previous iteration of the self-consistency cycle. Discarding for a moment the last term, the two residual blocks of Eq. (A.5) contain $(B\nu)^2$ and $2B\nu^2$ independent real equations respectively, exactly as many as the free real parameters of a Hermitian Λ_i^c and of a complex $\mathcal{D}_i$: the fit formally is a square nonlinear root-finding problem, which is however solved as a least-squares problem.

The minimization of the distance is carried out with the Gauss-Newton algorithm implemented in `scipy.optimize.least_squares` using the analytic Jacobian of the residuals and, similarly to `Fragment.update_self_energy`, the derivative of the matrix Fermi function Eq. (A.1) is evaluated as a Fréchet derivative. The cost of the fit is once again dominated by the diagonalization of $\mathcal{H}_i^{0,\mathrm{emb}}$ and is negligible compared to a single solution of the interacting embedding problem.

Similarly to the previous method, the last term of Eq. (A.5) is a proximal penalty that keeps the solution close to the previous iteration. It keeps the ordering and the phases of the ghost levels continuous along the self-consistency cycle and along parameter scans. It does not bias the converged result since the penalty vanishes together with its gradient if the saddle point equations are satisfied. In practice, α_{p} is taken in the range 10^{-8}–10^{-6} (argument `move_pen`, default 10^{-6}) and can be increased or decreased adaptively, if a given step fails to converge.

Finally, the fit is performed on a real parametrization of $(\Lambda_i^c, \mathcal{D}_i)$, obtained by packing them into a single real vector y: the real upper triangle of Λ_i^c including the diagonal, its strict imaginary upper triangle, and the real and imaginary parts of $\mathcal{D}_i$ (routines `pack_params` and `unpack_params`). Hermiticity of Λ_i^c is thus imposed by construction rather than as a constraint, and the least-squares solver operates on unconstrained real variables. When S_z is a good quantum number the whole procedure is applied independently to each spin block (`use_Sz=True`), which halves the total number of fitted parameters and reduces the cost of the fitting.

References

[1] A. Georges, G. Kotliar, W. Krauth and M. J. Rozenberg, *Dynamical Mean-Field Theory of Strongly Correlated Fermion Systems and the Limit of Infinite Dimensions*, Reviews of Modern Physics **68**(1), 13 (1996), doi:10.1103/RevModPhys.68.13.

[2] M. C. Gutzwiller, *Correlation of Electrons in a Narrow s Band*, Physical Review **137**(6A), A1726 (1965), doi:10.1103/PhysRev.137.A1726.

[3] N. Lanatà, T.-H. Lee, Y.-X. Yao and V. Dobrosavljević, *Emergent Bloch Excitations in Mott Matter*, Physical Review B **96**, 195126 (2017), doi:10.1103/PhysRevB.96.195126.

[4] T.-H. Lee, C. Melnick, R. Adler, N. Lanatà and G. Kotliar, *Accuracy of ghost-rotationally-invariant slave-boson theory for multiorbital Hubbard models and realistic materials*, Phys. Rev. B **108**, 245147 (2023), doi:10.1103/PhysRevB.108.245147.

[5] S. Giuli, T.-H. Lee, Y.-X. Yao, G. Kotliar, A. E. Ruckenstein, O. Gingras and N. Lanatà, *Unifying Variational and Dynamical Quantum Embedding: From Ghost Gutzwiller Approximation to Dynamical Mean-Field Theory* (2026), doi:10.48550/arXiv.2603.20559, 2603.20559.

[6] S. Smit, M. Bluschke, P. Moen, N. Heinsdorf, E. Zavatti, G. Bellomia, S. Giuli, S. K. Y. Dufresne, C. T. Suen, V. Zimmermann, C. Au-Yeung, S. Zhdanovich *et al.*, *Enhanced coherence and layer-selective charge order in a trilayer cuprate superconductor* (2025), 2506.01448.

[7] A. Kazemi-Moridani, S. Giuli, T.-H. Lee, A. M. S. Tremblay, M. Côté, N. Lanatà and O. Gingras, *Magnetic phases of the anisotropic triangular Hubbard model from the ghost-Gutzwiller approximation in the rotating spin-frame* (2026), 2604.24848.

[8] G. Bellomia, A. Amaricci and M. Capone, *Local classical correlations between physical electrons in Hubbard systems*, Physical Review B **113**(15) (2026), doi:10.1103/cqfh-mbwk.

[9] M. Caffarel and W. Krauth, *Exact diagonalization approach to correlated fermions in infinite dimensions: Mott transition and superconductivity*, Physical Review Letters **72**(10), 1545 (1994), doi:10.1103/PhysRevLett.72.1545.

[10] A. Amaricci, L. Crippa, A. Scazzola, F. Petocchi, G. Mazza, L. de Medici and M. Capone, *EDIpack: A parallel exact diagonalization package for quantum impurity problems*, Computer Physics Communications **273**, 108261 (2022), doi:10.1016/j.cpc.2021.108261.

[11] L. Crippa, I. Krivenko, S. Giuli, G. Bellomia, A. Kowalski, F. Petocchi, A. Scazzola, M. Wallerberger, G. Mazza, L. de Medici, G. Sangiovanni, M. Capone *et al.*, *Next-generation EDIpack: A Lanczos-based package for quantum impurity models featuring general broken-symmetry phases, flexible bath topologies and multi-platform interoperability*, SciPost Phys. Codebases p. 58 (2025), doi:10.21468/SciPostPhysCodeb.58.

[12] T. N. Dionne, A. Foley, M. Rousseau and D. Sénéchal, *Pyqcm: An open-source Python library for quantum cluster methods*, SciPost Phys. Codebases p. 23 (2023), doi:10.21468/SciPostPhysCodeb.23.

[13] P. Werner and A. J. Millis, *Hybridization expansion impurity solver: General formulation and application to Kondo lattice and two-orbital models*, Phys. Rev. B **74**, 155107 (2006), doi:10.1103/PhysRevB.74.155107.

[14] P. Werner, A. Comanac, L. de' Medici, M. Troyer and A. J. Millis, *Continuous-Time Solver for Quantum Impurity Models*, Phys. Rev. Lett. **97**, 076405 (2006), doi:10.1103/PhysRevLett.97.076405.

[15] K. Haule, *Quantum Monte Carlo impurity solver for cluster dynamical mean-field theory and electronic structure calculations with adjustable cluster base*, Phys. Rev. B **75**, 155113 (2007), doi:10.1103/PhysRevB.75.155113.

[16] E. Gull, A. J. Millis, A. I. Lichtenstein, A. N. Rubtsov, M. Troyer and P. Werner, *Continuous-time Monte Carlo methods for quantum impurity models*, Reviews of Modern Physics **83**, 349 (2011), doi:10.1103/RevModPhys.83.349.

[17] P. Seth, I. Krivenko, M. Ferrero and O. Parcollet, *TRIQS/CTHYB: A continuous-time quantum Monte Carlo hybridisation expansion solver for quantum impurity problems*, Computer Physics Communications **200**, 274 (2016), doi:https://doi.org/10.1016/j.cpc.2015.10.023.

[18] M. Wallerberger, A. Hausoel, P. Gunacker, A. Kowalski, G. Sangiovanni and K. Held, *w2dynamics: Local one- and two-particle quantities from dynamical mean field theory*, Computer Physics Communications **235**, 388 (2019), doi:10.1016/j.cpc.2018.09.007.

[19] C. Melnick, P. Sémon, K. Yu, N. D'Imperio, A.-M. Tremblay and G. Kotliar, *Accelerated impurity solver for DMFT and its diagrammatic extensions*, Computer Physics Communications **267**, 108075 (2021), doi:https://doi.org/10.1016/j.cpc.2021.108075.

[20] N. Kavokine, H. Lu, T. Ayral, M. Ferrero, N. Wentzell and O. Parcollet, *CTSEG: A segment picture quantum impurity solver based on TRIQS*, Journal of Open Source Software **10**(109), 7425 (2025), doi:10.21105/joss.07425.

[21] F. A. Wolf, A. Go, I. P. McCulloch, A. J. Millis and U. Schollwöck, *Imaginary-Time Matrix Product State Impurity Solver for Dynamical Mean-Field Theory*, Physical Review X **5**, 041032 (2015), doi:10.1103/PhysRevX.5.041032.

[22] M. Ganahl, M. Aichhorn, P. Thunström, K. Held, H. G. Evertz and F. Verstraete, *Efficient DMFT impurity solver using real-time dynamics with matrix product states*, Physical Review B **92**, 155132 (2015), doi:10.1103/PhysRevB.92.155132.

[23] R. Žitko and T. Pruschke, *Energy resolution and discretization artifacts in the numerical renormalization group*, Phys. Rev. B **79**, 085106 (2009), doi:10.1103/PhysRevB.79.085106.

[24] O. Parcollet, M. Ferrero, T. Ayral, H. Hafermann, I. Krivenko, L. Messio and P. Seth, *TRIQS: A Toolbox for Research on Interacting Quantum Systems*, Computer Physics Communications **196**, 398 (2015), doi:10.1016/j.cpc.2015.04.023.

[25] B. Bauer, L. D. Carr, H. G. Evertz *et al.*, *The ALPS project release 2.0: Open source software for strongly correlated systems*, Journal of Statistical Mechanics: Theory and Experiment **2011**(05), P05001 (2011), doi:10.1088/1742-5468/2011/05/P05001.

[26] Y. Yao, *CyGutz: CyGutz*, doi:10.5281/zenodo.14076 (2015).

[27] R. Adler, C. Melnick and G. Kotliar, *Portobello - Quantum embedding in correlated materials made accessible*, Computer Physics Communications **294**, 108907 (2024), doi:https://doi.org/10.1016/j.cpc.2023.108907.

[28] E. M. Makaresz, O. Gingras, T.-H. Lee, N. Lanatà, B. J. Powell and H. L. Nourse, *Accelerating dynamical mean-field theory convergence by preconditioning with computationally cheaper quantum embedding methods* (2026), 2601.16401.

[29] G. Van Rossum, F. L. Drake *et al.*, *Python reference manual*, vol. 111, Centrum voor Wiskunde en Informatica Amsterdam (1995).

[30] K. Martin and B. Hoffman, *Mastering CMake : a cross-platform build system : version 3.1* (2015).

[31] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus *et al.*, *Array programming with NumPy*, Nature **585**(7825), 357 (2020), doi:10.1038/s41586-020-2649-2.

[32] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett *et al.*, *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*, Nature Methods **17**, 261 (2020), doi:10.1038/s41592-019-0686-2.

[33] S. K. Lam, A. Pitrou and S. Seibert, *Numba: a LLVM-based Python JIT compiler*, In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15. Association for Computing Machinery, New York, NY, USA, ISBN 9781450340052, doi:10.1145/2833157.2833162 (2015).

[34] The HDF Group, *Hierarchical Data Format, version 5*.

[35] L. Dalcin and Y.-L. L. Fang, *mpi4py: Status update after 12 years of development*, Computing in Science & Engineering **23**(4), 47 (2021), doi:10.1109/MCSE.2021.3083216.

[36] J. D. Hunter, *Matplotlib: A 2D graphics environment*, Computing in Science & Engineering **9**(3), 90 (2007), doi:10.1109/MCSE.2007.55.

[37] G. Brandl and the Sphinx Developers, *Sphinx Documentation*, https://www.sphinx-doc.org/, Accessed: 2026-08-13.

[38] S. R. White, *Density matrix formulation for quantum renormalization groups*, Phys. Rev. Lett. **69**, 2863 (1992), doi:10.1103/PhysRevLett.69.2863.

[39] S. Giuli, H. Hasan, B. Kloss, M. S. Frank, T.-H. Lee, O. Gingras, Y.-X. Yao and N. Lanatà, *Linear Foundation Model for Quantum Embedding: Data-Driven Compression of the Ghost Gutzwiller Variational Space*, PRX Intelligence **1**, 013001 (2026), doi:10.1103/phk3-pvhm.

[40] J. J. Dongarra, J. Du Croz, S. Hammarling and I. S. Duff, *A set of level 3 basic linear algebra subprograms*, ACM Trans. Math. Softw. **16**(1), 1–17 (1990), doi:10.1145/77626.79170.

[41] E. Anderson *et al.*, *LAPACK Users' Guide*, Society for Industrial and Applied Mathematics, Philadelphia, PA, third edn., ISBN 0-89871-447-8 (paperback) (1999).

[42] J. Kuneš, *Efficient treatment of two-particle vertices in dynamical mean-field theory*, Phys. Rev. B **83**, 085102 (2011), doi:10.1103/PhysRevB.83.085102.

[43] A. Georges, L. d. Medici and J. Mravlje, *Strong Correlations from Hund's Coupling*, Annual Review of Condensed Matter Physics **4**(Volume 4, 2013), 137 (2013), doi:https://doi.org/10.1146/annurev-conmatphys-020911-125045.

[44] N. Lanatà, H. U. R. Strand, G. Giovannetti, B. Hellsing, L. de' Medici and M. Capone, *Orbital selectivity in Hund's metals: The iron chalcogenides*, Phys. Rev. B **87**, 045122 (2013), doi:10.1103/PhysRevB.87.045122.

[45] C. Mejuto-Zaera and M. Fabrizio, *Efficient computational screening of strongly correlated materials: Multiorbital phenomenology within the ghost Gutzwiller approximation*, Phys. Rev. B **107**, 235150 (2023), doi:10.1103/PhysRevB.107.235150.

[46] S. Giuli, R. J. Campos-Lopes, E. Moghadas and M. Capone, *Ghost-RISB for Correlated Electron-Phonon Systems: Application to the Hubbard-Holstein Model* (2026), 2608.04816.

[47] H. LaBollita *et al.*, *TRIQS/ModEST: Modular electronic structure toolkit*, In preparation (2026).

[48] T.-H. Lee, C. Melnick, R. Adler, X. Sun, Y. Yao, N. Lanatà and G. Kotliar, *Charge self-consistent density functional theory plus ghost rotationally invariant slave-boson theory for correlated materials*, Phys. Rev. B **110**, 115126 (2024), doi:10.1103/PhysRevB.110.115126.

[49] Wikipedia contributors, *Fréchet derivative*, *Wikipedia, The Free Encyclopedia*, Accessed: 2026-09-14 (2026).

[50] M. Carlsson, *A daleskii–krein theorem for hermitian matrix-functions based on vector-fields* (2019), doi:10.48550/arXiv.1811.08358, 1811.08358.